



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

AGOSTO-DICIEMBRE 2025

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES / INFORMATICA

MATERIA Y SERIE :

PATRONES DE DISEÑO

TÍTULO:

EXAMEN

UNIDAD A EVALUAR:

2DA UNIDAD

NOMBRES Y NÚMEROS DE CONTROL DE LOS ALUMNOS:

ZARZA MORALES JOSE DIEGO 22210366

NOMBRE DEL MAESTRO:

MARIBEL GUERRERO LUIS

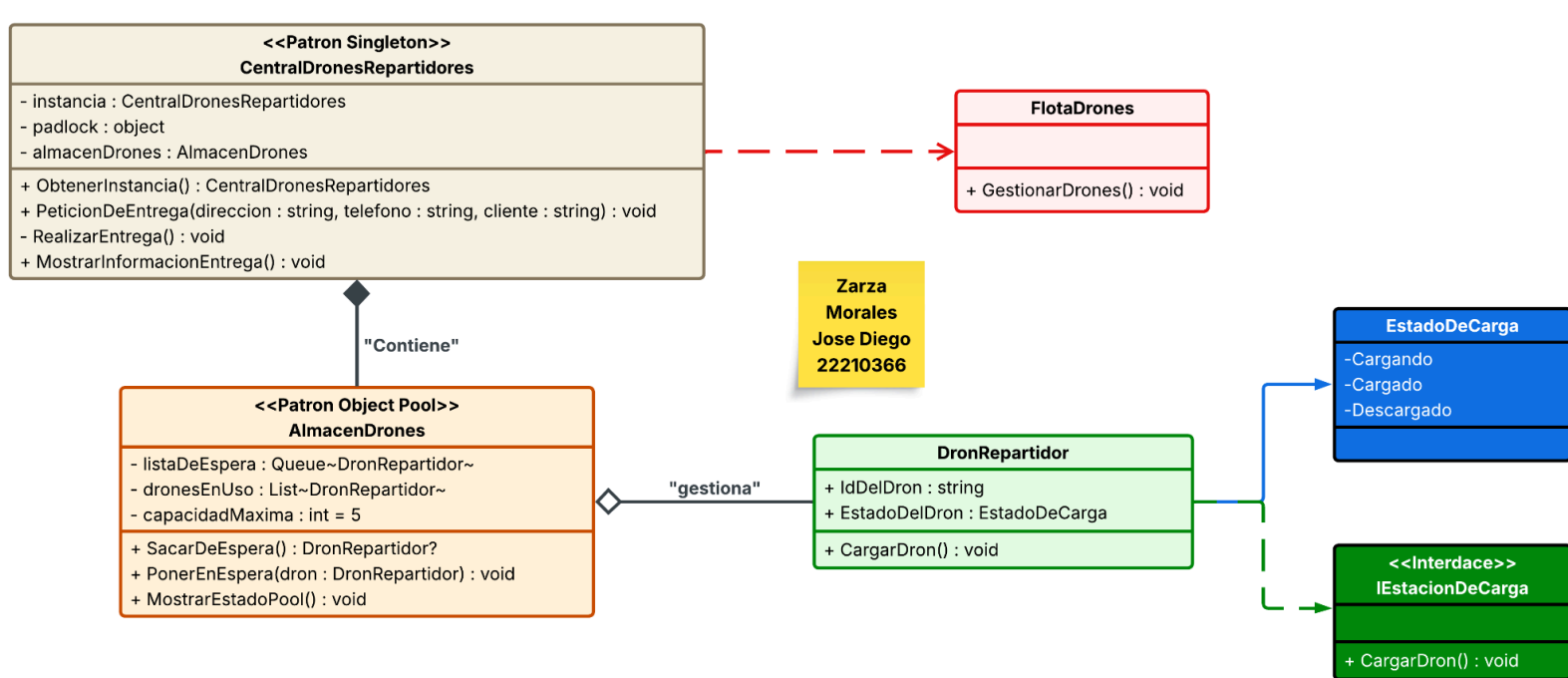
FECHA:

15 DE OCTUBRE DEL 2025

Índice

1.- Diagrama UML.....	3
2.- Código.....	4
3.-Captura de pantalla de su programa funcionando.....	9
4.-Conclusión.....	10

1.- Diagrama UML



2.- Código

```
using System;
using CentralDronesRepartidores;

namespace FlotaDeDronesRepartidores
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Sistema de Entrega por Drones ===\n");

            var flotaDrones = new FlotaDrones();
            flotaDrones.GestionarDrones();

            Console.WriteLine("\nPresione cualquier tecla para salir...");
            Console.ReadKey();
        }
    }
}
```

```
using System;
using AlmacenamientoDronesRepartidores;

namespace CentralDronesRepartidores
{
    // Implementación del patrón Singleton para la central de drones
    // Garantiza una única instancia que coordina todas las entregas
    public class CentralDronesRepartidores
    {
        // Instancia única de la clase (Singleton)
        private static CentralDronesRepartidores? instancia;
        // Objeto de bloqueo para garantizar thread-safety
        private static readonly object padlock = new object();
        // Referencia al pool de drones (Object Pool)
        private readonly AlmacenDrones almacenDrones;
    }
}
```

```
public string DireccionEntrega { get; private set; }
public string TelefonoEntrega { get; private set; }
public string NombreDelCliente { get; private set; }

// Constructor privado donde se realiza la conexión con el Object Pool
// Al ser un Singleton, se asegura que solo exista una instancia del
AlmacenDrones (Object Pool)
// Esta es la conexión principal entre los dos patrones de diseño
private CentralDronesRepartidores()
{
    // Aquí se crea la única instancia del Object Pool que existirá en el
sistema

    almacenDrones = new AlmacenDrones();
    DireccionEntrega = string.Empty;
    TelefonoEntrega = string.Empty;
    NombreDelCliente = string.Empty;
}

// Método Singleton para obtener la única instancia de la central
public static CentralDronesRepartidores ObtenerInstancia()
{
    if (instancia == null)
    {
        lock (padlock)
        {
            if (instancia == null)
            {
                instancia = new CentralDronesRepartidores();
            }
        }
    }
    return instancia;
}

// Método público para iniciar una nueva entrega
// Recibe los detalles del pedido y coordina la entrega
public void PeticionDeEntrega(string direccion, string telefono, string
cliente)
{
    DireccionEntrega = direccion;
    TelefonoEntrega = telefono;
}
```

```
        NombreDelCliente = cliente;

        RealizarEntrega();
    }

    // Método privado que maneja la lógica de entrega
    // Gestiona la obtención del dron, simulación de entrega y devolución al pool
    private void RealizarEntrega()
    {
        Console.WriteLine("\n=== Nueva Entrega ===");
        MostrarInformacionEntrega();

        // Obtiene un dron del Object Pool
        var dron = almacenDrones.SacarDeEspera();
        if (dron != null)
        {
            Console.WriteLine($"Asignando {dron.IdDelDron} para la entrega...");
            Console.WriteLine("El dron está en camino al destino");
            System.Threading.Thread.Sleep(3000); // Simulamos el tiempo de entrega

            Console.WriteLine("\nEntrega completada!");
            almacenDrones.PonerEnEspera(dron); // Devuelve el dron al
almacen/PoolDeObjetos
        }
        else
        {
            Console.WriteLine("No se pudo realizar la entrega: no hay drones
disponibles.");
        }

        almacenDrones.MostrarEstadoPool();
    }

    public void MostrarInformacionEntrega()
    {
        Console.WriteLine("\nInformacion de Entrega:");
        Console.WriteLine($"Direccion: {DireccionEntrega}");
        Console.WriteLine($"Telefono: {TelefonoEntrega}");
        Console.WriteLine($"Nombre del Cliente: {NombreDelCliente}");
    }
}
```

```
public class FlotaDrones
{
    public void GestionarDrones()
    {
        var central = CentralDronesRepartidores.ObtenerInstancia();

        // Simular varias entregas
        central.PeticionDeEntrega("Xicotencatl Leyva", "664 245 2535", "Diego Zarza");

        central.PeticionDeEntrega("Urbis 2", "664 345 123", "Kevin Alvarez");
        central.PeticionDeEntrega("Cacho", "664 123 456 ", "Jordhy Rojas");
        central.PeticionDeEntrega("Salvatierra", "664 903 923", "Marvin Morales");
        central.PeticionDeEntrega("Natura", "664 882 023", "Jose Morales");

        //Aquí ponemos otra entrega para ver que no generamos otro dron y usamos un dron listo de el almacen(la piscina de objetos)
        central.PeticionDeEntrega("Riberas del bosque", "664 185 0613", "Juan Zarza");
    }
}
```

```
using System;
using System.Collections.Generic;

namespace AlmacenamientoDronesRepartidores
{
    public class EstacionDeCargaDeDronesRepartidores
    {
        public interface IEstacionDeCarga
        {
            //La carga del dron es como el reinicio para poder mandarlo al Object Pool
            void CargarDron();
        }
    }

    //Aquí hago lo mismo que con las habitaciones, les ponemos estados a los drones para poder mandarlos al object pool
}
```

```
public enum EstadoDeCarga
{
    Cargando,
    Cargado,
    Descargado
}

// Clase que representa cada dron individual y su capacidad de carga
public class DronRepartidor : EstacionDeCargaDeDronesRepartidores.IEstacionDeCarga
{
    public string IdDelDron { get; set; }
    public EstadoDeCarga EstadoDelDron { get; set; }

    public DronRepartidor(string id)
    {
        IdDelDron = id;
        EstadoDelDron = EstadoDeCarga.Descargado;
    }

    public void CargarDron()
    {
        Console.WriteLine($"{IdDelDron} esta siendo cargado.");
        EstadoDelDron = EstadoDeCarga.Cargando;

        System.Threading.Thread.Sleep(2000);
        EstadoDelDron = EstadoDeCarga.Cargado;
        Console.WriteLine($"{IdDelDron} esta completamente cargado.");
    }
}

// Object Pool: Gestiona un grupo de drones reutilizables
// Implementación del patrón Object Pool para gestionar drones reutilizables
public class AlmacenDrones
{
    private Queue<DronRepartidor> listaDeEspera;
    private List<DronRepartidor> dronesEnUso;
    private static readonly int capacidadMaxima = 5;

    public AlmacenDrones()
    {

```



```
listaDeEspera = new Queue<DronRepartidor>();
dronesEnUso = new List<DronRepartidor>();

// Inicializar el pool con drones
for (int i = 1; i <= capacidadMaxima; i++)
{
    var nuevoDron = new DronRepartidor($"Dron-{i}");
    listaDeEspera.Enqueue(nuevoDron);
}

public DronRepartidor? SacarDeEspera()
{
    if (listaDeEspera.Count == 0)
    {
        Console.WriteLine("No hay drones disponibles en este momento.");
        return null;
    }

    var dron = listaDeEspera.Dequeue();
    dronesEnUso.Add(dron);
    return dron;
}

public void PonerEnEspera(DronRepartidor dron)
{
    if (dron != null && dronesEnUso.Contains(dron))
    {
        dronesEnUso.Remove(dron);
        dron.CargarDron(); // Recargar el dron antes de ponerlo disponible
        listaDeEspera.Enqueue(dron);
        Console.WriteLine($"{dron.IdDelDron} ha sido devuelto a la estacion.");
    }
}

public void MostrarEstadoPool()
{
    Console.WriteLine($"\\nEstado de la estacion de Drones:");
    Console.WriteLine($"Drones en espera: {listaDeEspera.Count}");
    Console.WriteLine($"Drones en uso: {dronesEnUso.Count}");
}
```



3.-Captura de pantalla de su programa funcionando

Iniciamos el programa, y aquí desplegamos la información de entrega para el dron, la cual tiene dirección, teléfono y nombre del cliente.

Asignamos el dron, aquí el primer dron es el primero en salir, tenemos un tiempo de simulación de que el dron anda entregando.

Ya al terminar de entregar el dron regresa al almacén y va a la estación de carga, porque obviamente viene muy cansado(descargado) de entregar jeje, ya cuando el dron se cargue pues lo mandamos al almacén el cual es nuestra piscina de objetos, aquí aplique lo mismo que hicimos con las habitaciones, le puse estados al dron, cargado, descargado y cargado, cuando el dron este cargado pues lo mandamos al almacén con el método PonerEnEspera().

Y después de esto despliego lo que es el estado de la estación de drones, esto es para ver que no creamos un dron nuevo, nuestro máximo de drones son 5. ahí tenemos lo que es drones en espera y drones en uso. pero como yo directamente cargo al dron después de entregar y lo paso a su almacén pues tendríamos ese dron ya en el almacenamiento.

```
PS C:\Users\josez\Documents\PracticasPatronesDeDisenoDZ> cd Examen
PS C:\Users\josez\Documents\PracticasPatronesDeDisenoDZ\Examen> dotnet run
=== Sistema de Entrega por Drones ===

=== Nueva Entrega ===

Informacion de Entrega:
Direccion: Xicotencatl Leyva
Telefono: 664 245 2535
Nombre del Cliente: Diego Zarza

Asignando Dron-1 para la entrega...
El dron está en camino al destino

Entrega completada!
Dron-1 esta siendo cargado.
Dron-1 esta completamente cargado.
Dron-1 ha sido devuelto a la estacion.

Estado de la estacion de Drones:
Drones en espera: 5
Drones en uso: 0
```

Dron-2 y Dron-3 Entregando

```
=== Nueva Entrega ===

Informacion de Entrega:
Direccion: Urbis 2
Telefono: 664 345 123
Nombre del Cliente: Kevin Alvarez

Asignando Dron-2 para la entrega...
El dron está en camino al destino

Entrega completada!
Dron-2 esta siendo cargado.
Dron-2 esta completamente cargado.
Dron-2 ha sido devuelto a la estacion.

Estado de la estacion de Drones:
Drones en espera: 5
Drones en uso: 0

=== Nueva Entrega ===

Informacion de Entrega:
Direccion: Cacho
Telefono: 664 123 456
Nombre del Cliente: Jordhy Rojas

Asignando Dron-3 para la entrega...
El dron está en camino al destino

Entrega completada!
Dron-3 esta siendo cargado.
Dron-3 esta completamente cargado.
Dron-3 ha sido devuelto a la estacion.

Estado de la estacion de Drones:
Drones en espera: 5
Drones en uso: 0
```

Zarza Morales Jose Diego 22210366

Dron-4 y Dron-5 Entregando

=== Nueva Entrega ===

Informacion de Entrega:

Direccion: Salvatierra

Telefono: 664 903 923

Nombre del Cliente: Marvin Morales

Asignando Dron-4 para la entrega...

El dron está en camino al destino

Entrega completada!

Dron-4 esta siendo cargado.

Dron-4 esta completamente cargado.

Dron-4 ha sido devuelto a la estacion.

Estado de la estacion de Drones:

Drones en espera: 5

Drones en uso: 0

=== Nueva Entrega ===

Informacion de Entrega:

Direccion: Natura

Telefono: 664 882 023

Nombre del Cliente: Jose Morales

Asignando Dron-5 para la entrega...

El dron está en camino al destino

Entrega completada!

Dron-5 esta siendo cargado.

Dron-5 esta completamente cargado.

Dron-5 ha sido devuelto a la estacion.

Estado de la estacion de Drones:

Drones en espera: 5

Drones en uso: 0

Zarza Morales Jose Diego 22210366

Como en el código agregue una sexta entrega muestra que ya lo hace el Dron-1 y no otro dron

```
Drones en uso: 0

=== Nueva Entrega ===

Informacion de Entrega:
Direccion: Riberas del bosque
Telefono: 664 185 0613
Nombre del Cliente: Juan Zarza

Asignando Dron-1 para la entrega...
El dron está en camino al destino

Entrega completada!
Dron-1 esta siendo cargado.
Dron-1 esta completamente cargado.
Dron-1 ha sido devuelto a la estacion.

Estado de la estacion de Drones:
Drones en espera: 5
Drones en uso: 0

Presione cualquier tecla para salir...
```

4.-Conclusión

Usar estos dos patrones es bueno por que aqui hacemos que nuestra central sea la que controle los drones y cuando ya no los ocupe los manda a su almacén, y ahí se quedan hasta que los necesiten, así como los videojuegos, lo bueno de tener estos patrones juntos es que se dividen las responsabilidades y se valida cuando los drones se guardan y salen de su almacenamiento.